Set 1 · Exercise 4

New Claims Intake → Claims Register Data Table

Industry: Insurance | **Tools:** Google Sheets · Lookup Table · Workato Data Table **Estimated time:** 90–120 minutes

---

What You Will Learn

- **Workspace Management** — setting up the correct folder structure and naming conventions before building
- **Developing Recipes** — building a scheduler-triggered recipe with conditional logic, Lookup Table data enrichment, and bidirectional sheet writeback
- **Working with JSON Data** — parsing a nested JSON field using the Parse JSON action and mapping the output datapills
- **Function Calls** — understanding why For Each loop processing is handled inside a function call, keeping the main recipe clean

---

Scenario

SecureLife Insurance's claims intake team logs every new claim submission in a shared Google Sheet called **SecureLife_Claims_Intake**. The claims operations team needs all valid **Submitted** claim records stored in a centralised Workato Data Table called **Claims_Register** so that adjusters, compliance, and billing teams can draw from a single source of truth.

Build a recipe that validates each new row, pulls in adjuster assignment data from a Lookup Table, parses incident details from a JSON field, creates a row in the Data Table, and writes the outcome back to the source sheet.

Workspace Setup

Set up the following folder and asset structure before building your recipe. Follow the naming format from the Workato Best Practices Guide.

```
[SecureLife] Claims Operations  (project root)
│
├── Lookup Tables
│   └── [SecureLife] LT-001 | Policy Type Directory
│
└── Claims Intake
    ├── Recipes
    │   └── [SecureLife] REC-001 | New Claims Intake to Claims Register
    └── Functions
        └── [SecureLife] FUN-001 | Process Claim Row
```

Trigger

> Scheduler — Every day at **9:30 AM IST**

Recipe Steps

Main recipe — [SecureLife] REC-001

1. **Trigger** — Scheduler: Every day at 9:30 AM IST
2. **Action** — Google Sheets: Get all rows from SecureLife_Claims_Intake, Sheet1
3. **Action** — Repeat for each row in the rows list
4. **Action (inside loop)** — Call function: [SecureLife] FUN-001 | Process Claim Row — pass all row field datapills + Row_Number as inputs

> All processing logic lives inside the function. The main recipe only fetches rows and loops. This follows the Best Practices Guide: *"Create a function for all steps inside a loop to retain all historic data."* Keeping the loop body in a function ensures each row's job is logged independently in the audit trail, making it easy to debug individual failures without sifting through the entire batch job.

Function recipe — [SecureLife] FUN-001

5. **Condition** — IF Claim_Status = `Submitted` AND Claimant_Email is present AND Incident_Date is present → **continue to steps 7–11**

6. **ELSE** — Google Sheets: Update same row → Sync_Status = `❌ Skipped`, Remarks = reason → Stop

> Check conditions in the order listed above. Write the reason for the **first** failing condition only.

Failing condition	Remarks value to write
Claim_Status ≠ Submitted	`Skipped: Claim status is not Submitted`
Claimant_Email is empty	`Skipped: Claimant email is missing`
Incident_Date is empty	`Skipped: Incident date is missing`

7. **Action** — Parse JSON document
   - **Document:** `Claim_Details` datapill from the loop
   - **Sample document:** paste the JSON structure below — Workato uses this to generate output datapills

```json
{
  "incident": {
    "location": "Highway 45",
    "description": "Rear-end collision"
  },
  "estimated_loss_INR": 85000,
  "witness_available": "Yes"
}
```

This step converts the raw JSON string into usable datapills. You will see `location` and `description` (nested under `incident`), `estimated_loss_INR`, and `witness_available` appear as output datapills from this step.

8. **Action** — Lookup Table: Look up Policy_Type → returns Assigned_Adjuster + Coverage_Limit_INR

> **Tip:** Workato has a `lookup()` formula that lets you query a Lookup Table directly inside a field without adding a separate step. This saves tasks since no additional action step is consumed. Try replacing this step with the lookup formula once your recipe is working.
> **Syntax:**
> ```
> lookup("TABLE_NAME", "REFERENCE_COLUMN": datapill)["LOOKUP_COLUMN"]
> ```

9. **Action** — Data Table: Create row in Claims_Register — map fields using datapills from Step 7 (parsed JSON), Step 8 (Lookup Table), and the loop row
10. **Action (Success)** — Google Sheets: Update same row → Sync_Status = ✅ `Synced`, Data_Table_Record_ID = record ID from Step 9
11. **Action (On Error)** — Google Sheets: Update same row → Sync_Status = ⚠ `Failed`, Remarks = error message

---

Data Fields

Data Table: Claims_Register (destination)

Column	Type
Claim_ID	String
Claimant_First_Name	String
Claimant_Last_Name	String
Claimant_Email	String
Policy_Type	String
Policy_Number	String
Incident_Date	String
Incident_Location	String
Incident_Description	String
Estimated_Loss_INR	String
Witness_Available	String
Assigned_Adjuster	String
Coverage_Limit_INR	String
Intake_Timestamp	String

Writeback columns (recipe writes back to sheet)

Column	Written on
Sync_Status	All paths — ✅ Synced / ❌ Skipped / ⚠ Failed
Data_Table_Record_ID	Success path only
Remarks	Skip path (reason) and Error path (error message)

Expected Outcome

After the recipe runs, the Google Sheet should show a Sync_Status for every row — no row should be blank. The Claims_Register Data Table should contain exactly **6 rows**.

Best Practices Checklist

Before submitting, make sure the following are in place:

- ☐ Error handling block
- ☐ Step comments
- ☐ Job tabs
- ☐ Naming conventions followed